SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This System Analysis Documentation introduces the technical design of LRT AI Operations - the strategic solution to the problem identified in the Product Requirement Documentation, namely that Malaysian LRT duty managers make slow, inconsistent, manual scheduling decisions during disruptions. The system uses Z.AI's GLM (ilmu-glm 5.1) as a reasoning service layer on top of a deterministic operational math core to deliver real-time, justified scheduling recommendations.

1.1. Purpose:

This document covers the technical scope and design decisions behind the LRT AI Operations system: how the architecture is partitioned, how data flows from frontline staff input through GLM reasoning to the persisted schedule, and how each component will be built and validated within the hackathon timeline.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *A monolithic Streamlit web application with a clearly partitioned internal package structure (UI → orchestration → math core / GLM client / persistence). The app calls Z.AI's GLM endpoint as an external service and uses local SQLite for state. The architecture is intentionally simple for a hackathon MVP, but the layering is service-oriented so any module (e.g. database, GLM client) can be swapped or extracted into a microservice in production.*
2. ***Data Flows:***
    a. *Situation data flows from the duty manager's input form or free-text through the GLM extraction layer (with OCR for images), into the deterministic math layer that computes three scheduling options, then into the GLM recommendation layer that picks one and explains why, and finally into the SQLite persistence layer when the manager applies and saves the chosen schedule.*
3. ***Model Process:***
    a. *The model shows the end-to-end workflow of the four core features: multi-modal situation capture, three-option trade-off analysis, GLM-powered recommendation with streamed reasoning, and schedule application with audit-trail persistence.*
4. ***Role of Reference:***
    a. *This document is the technical reference for all developers, testers, and reviewers evaluating the LRT AI Operations system, and is the source of truth for how GLM is integrated, where its limits are enforced, and how the system degrades gracefully if GLM is unavailable.*

© UMHackathon 2026

1.2. Background:

Malaysian LRT duty managers currently make schedule adjustments manually using fragmented sources: paper or spreadsheet ridership tables, weather forecasts, externally-published event schedules, internal incident logs, and personal experience. There is no unified tool that fuses this data with reasoning capability, and no system of record that captures why a given adjustment was made. The cost and service consequences of manual decisions are significant - each unnecessary train costs RM 350 per train-hour (example), and a single major event can produce thousands of stranded passengers if under-provisioned.

1. *Previous Version:*
    a. *There is no prior software version of this product. The status quo is a fully manual process performed by duty managers using operational binders and the standard timetable spreadsheet. The MVP described in this document is therefore version 1.0.*
2. ***Changes in Major Architectural Components:*** *Compared to the manual baseline, the new system introduces three architectural components that did not previously exist: (1) a deterministic math core that pre-computes three differentiated scheduling options with full quantitative metrics; (2) a GLM reasoning service that interprets unstructured situation input and selects/justifies one option; (3) a SQLite-backed schedule store keyed by (date, line) that turns each decision into a permanent, auditable record. A lightweight retrieval layer reads historical precedents from a JSON store and injects the most relevant past incident into GLM's prompt.*
3. ***New capabilities introduced in this version:*** *multi-modal situation capture (manual / free-text), live-streamed GLM reasoning visible to the duty manager as it is generated, deterministic safety overrides for track-incident and signal-failure cases (zero trains regardless of GLM output), automatic shift-briefing generation for handover, and a weekly cost overview comparing standard versus adjusted operating spend.*

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
LRT Duty Manager (primary)	*Decides hourly train frequency in real time when disruptions occur; describes situations and applies recommendations.*	*Decision in under 30 seconds, plain-language reasoning, ability to override the AI pick, and full audit trail of their actions.*

© UMHackathon 2026

OCC Supervisor	*Oversees decisions across multiple lines and approves major schedule changes.*	*Visibility into all adjusted schedules, ability to retrieve any past decision by date and line, and confidence that safety overrides cannot be bypassed.*
Shift Handover Staff	*Takes over the duty seat at shift change and continues operating from the current schedule.*	*Auto-generated shift briefing under 220 words, saved schedule visible immediately on login, no need to re-run analysis.*
Operations Planner	*Reviews historical adjustments to refine standard timetables and event playbooks.*	*Searchable history of past situations and chosen options, ability to compare predicted versus actual cost outcomes.*
Development Team	*Builds and maintains the system across MVP and post-hackathon iterations.*	*Clear module boundaries (UI / orchestration / math core / GLM client / persistence), well-documented prompt contracts, and deterministic fallback paths for testing.*
QA / Validation Team	*Validates system behaviour across normal, edge, and emergency scenarios.*	*Defined scenarios (weather × event × emergency combinations), reproducible math outputs given fixed inputs, and clearly-labelled fallback paths to test independently.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web application (desktop browser, ops-room workstation)*
Architecture	*Layered monolith with external GLM service — Streamlit UI on top of an internal core package (orchestrator + math layer + GLM client + database), calling Z.AI GLM API as external services.*

LRT AI Operations is structured as a single-tier web application with a layered internal core. The UI tier is a Streamlit app (app.py) running in the duty manager's browser. Below it sits an orchestration module (core/recommender.py) that coordinates the math core (core/calculator.py), the GLM client (core/glm_client.py), and the persistence layer (core/database.py, SQLite). The system talks to an external service: Z.AI's GLM API for reasoning. The math core is the deterministic backbone - it computes the three scheduling options and enforces safety overrides regardless of GLM output. The GLM client is the reasoning service layer that handles prompt construction, streaming, response parsing, and graceful degradation.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

2.1.3 Dependency Diagram

a. ***Prompt construction:*** *Each GLM call has a dedicated system prompt embedded in core/recommender.py and core/glm_client.py. The recommender prompt assembles the situation summary (line, datetime, weather, events, emergency), the three pre-computed options with their metrics, and one historical precedent retrieved from data/history.json. The extraction prompt assembles a strict JSON-schema instruction plus the raw user text.*

b. ***Context window contents:*** *Three structured blocks are sent to GLM per recommendation call: (1) SITUATION SUMMARY (line, day, weather, events, emergency); (2) THREE SCHEDULING OPTIONS (with load factor, headway, passengers served, cost delta); (3) HISTORICAL PRECEDENTS (one record matched by line/weather/event). Total input is approximately 1,500–2,500 tokens - well within ilmu-glm 5.1's limit and bounded by design.*

© UMHackathon 2026

c. ***Response handling:*** *Responses are streamed via Server-Sent Events. The call_glm_stream generator in core/glm_client.py yields text deltas which Streamlit renders live with st.write_stream. After streaming completes, _parse_recommendation in core/recommender.py extracts the trailing "RECOMMENDATION: <choice>" tag and separates explanation from choice. The choice (conservative / moderate / aggressive) drives the math core to apply the corresponding frequency delta to the schedule.*

d. ***Token limits and chunking:*** *Free-text descriptions are truncated to 2,000 characters before being sent to GLM. Historical precedents in the recommendation prompt are capped at one record. These limits are enforced in core/glm_client.py (image compression, text truncation) and core/recommender.py (history retrieval cap), before the request hits the GLM API.*

e. ***Major API calls between components:*** *Streamlit UI → orchestrator (recommender.recommend_daily / get_glm_recommendation_stream); orchestrator → math core (calculator.compute_options, calculator.compute_daily_schedule); orchestrator → GLM client (call_glm_stream over HTTPS to api.z.ai/api/paas/v4/chat/completions) → UI → persistence layer (database.save_schedule / load_schedule / list_saved over SQLite).*

f. ***Frontend ↔ backend ↔ services ↔ database ↔ external API:*** *Streamlit (frontend) renders directly from session_state populated by the orchestrator (backend). The orchestrator writes the saved schedule to SQLite via the database module to ensure persistence. The GLM client is the only module that touches Z.AI's external API - keeping external dependencies isolated for testing and replacement.*

g. ***Distributed deployment plan:*** *The hackathon MVP is deployed as a single process. For production, the same module boundaries support extracting the GLM client into a stateless reasoning microservice, the database into a managed Postgres or MySQL instance, and the math core into a shared library used by both real-time and batch schedulers - all without changes to the orchestrator's public interfaces.*

2.2.2. Sequence Diagram

a. ***Primary user flow — Adjust Schedule For Concert + Rain:*** *Duty manager opens the app at 14:00, picks tomorrow's date and Kelana Jaya line; standard schedule loads from the math core. Manager selects 'Describe in text' and types "Coldplay concert at Bukit Jalil from 7pm, expecting 5,000 extra pax/hr, weather forecast rainy." Manager clicks Analyse.*

b. ***Step-by-step interactions:*** *(1) UI sends raw text to glm_client.extract_inputs_from_text. (2) GLM returns structured JSON: weather=rainy, event={name:'Coldplay', pax/hr:5000, type:'concert'}. (3) UI calls calculator.compute_options, which computes conservative/moderate/aggressive options for the representative hour using rainy multiplier and event surge. (4) UI displays the three option cards instantly. (5) UI calls recommender.get_glm_recommendation_stream; GLM streams its reasoning live,*

ending with "RECOMMENDATION: aggressive" because rain raises event LRT use and load factor would otherwise spike. (6) Manager clicks Apply; calculator.compute_daily_schedule rebuilds the full 06:00–24:00 schedule with the chosen delta applied to the event window. (7) Manager clicks Save; database.save_schedule writes the row to lrt_schedules.db.

c. ***End-to-end demonstration:*** *The same sequence diagram covers the two input modes (manual, text) by varying only the first step: manual inputs bypass GLM extraction; text and manual paths converge at compute_options. The emergency path adds a deterministic override step between compute_options and the option cards - for track_incident or signal_failure, the math core sets frequency to zero and the UI displays a red safety banner before GLM finishes streaming.*

2.2. Technological Stack

2.2.1. ***Frontend:*** *Streamlit 1.30+ — chosen for rapid iteration on data-heavy UIs with built-in support for live streaming output (st.write_stream), session state management, dataframes, and conditional row styling, all without writing custom HTML/CSS/JS.*

2.2.2. ***Backend:*** *Python 3.12 modules organised as a layered core package (core/recommender, core/calculator, core/glm_client, core/database). The orchestrator (recommender) is the only module the UI imports directly; it composes the math layer, GLM client, and database as services.*

2.2.3. ***Database:*** *SQLite (file-based, lrt_schedules.db) — chosen for the MVP because the data volume is small (one row per (date, line) decision), it requires zero infrastructure to demo, and the schema can be migrated to Postgres post-hackathon without any code changes beyond the connection string.*

2.2.4. ***Cloud / Deployment:*** *MVP runs locally during the hackathon demo and is deployable to Streamlit Community Cloud or any container host (Render, Fly.io, AWS ECS) with one Dockerfile. External service: Z.AI GLM API (reasoning). API keys are loaded from a .env file via python-dotenv — never hard-coded.*

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a. ***Data movement across the system:*** *External entity DUTY MANAGER provides three input streams (form / text / image) to process P1 SITUATION CAPTURE; P1 outputs structured factors to process P2 OPTIONS COMPUTATION, which reads ridership baselines and weather/event multipliers from data store D1 (in-memory constants) and history records from data store D2 (history.json).*

© UMHackathon 2026

P2 outputs three options to process P3 GLM RECOMMENDATION (calling external service Z.AI GLM); P3 outputs the chosen option and reasoning to process P4 SCHEDULE APPLICATION; P4 writes the final schedule to data store D3 (lrt_schedules.db) and back to the duty manager.

b. ***Black hole / DFD law compliance:*** *Every process has both inputs and outputs (no black holes); every data store is read by at least one process and written by at least one process; external entities only communicate with processes, never directly with data stores; data flows are labelled with their content (raw_text, extracted_factors, options[3], chosen_option, schedule_row).*

2.3.2. **Normalized Database Schema**

a. *The MVP's persistent schema is intentionally compact: a single SAVED_SCHEDULES table keyed by (date, line) holds one row per adjusted day. Embedded JSON columns (schedule_json, events_json) store the full hourly schedule and event list as denormalized documents because they are always read and written together as a unit - splitting them into separate tables would add joins with no integrity benefit. For the post-hackathon production schema, the ERD normalizes to 3NF with: SAVED_SCHEDULE (date, line, weather, emergency_type, total_std_cost, total_extra_cost, total_cost, saved_at), SCHEDULE_HOUR (schedule_id FK, hour, standard_freq, recommended_freq, expected_pax, load_factor, extra_cost), and EVENT (schedule_id FK, name, start_hour, end_hour, pax_per_hr, event_type) - with EVENT and SCHEDULE_HOUR both referencing SAVED_SCHEDULE as parent.*

3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. *Minimum Viable Product:*

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. ***[e.g. in Table]***

#	*Features*	*Description*
1	*Multi-Modal Situation Capture*	*Duty manager describes the situation via manual form or free-text. GLM extracts structured factors (weather, event, emergency, line) from any input mode.*
2	*Three-Option Trade-Off Analysis*	*Math core instantly computes three differentiated scheduling options (conservative, moderate, aggressive) with full metrics: load factor, headway,*

© UMHackathon 2026

		passengers served, cost delta. Options are spread around the optimal frequency for the situation.
3	*GLM Recommendation with Streamed Reasoning*	*GLM reads situation + three options + one historical precedent, picks one option, and streams its reasoning live to the duty manager. A trailing tag ("RECOMMENDATION: <choice>") drives the schedule application.*
4	*Schedule Application & Audit Persistence*	*One-click apply rebuilds the full daily schedule with the chosen frequency delta over the affected window; deterministic safety overrides (zero trains for track_incident / signal_failure) cannot be bypassed. Saved to SQLite keyed by (date, line) for full audit trail.*
5	*Auto-Generated Shift Briefing*	*After every applied adjustment, GLM auto-generates a sub-220-word briefing for the duty manager covering what to action, when, and why - ready for shift handover. Briefing is cached in session state to avoid rebilling on page reload.*

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*System must handle multiple duty managers concurrently across the three lines during peak event seasons (Merdeka, major concerts).*	*Stateless orchestrator and GLM client allow horizontal scaling behind a load balancer; SQLite is replaced with managed Postgres for production; session state is moved from Streamlit-process memory to Redis.*
Reliability	*Critical safety paths (track_incident, signal_failure) must produce zero-train output 100% of the time, and applied schedules must never be lost once Save returns success.*	*Deterministic safety overrides hard-coded in calculator.compute_options; INSERT OR REPLACE on (date, line) primary key with synchronous SQLite commit; explicit error display on save failure.*

© UMHackathon 2026

Maintainability	*Each module (UI, orchestrator, math core, GLM client, database) must be testable and replaceable independently. Prompts must be editable without touching application logic.*	*Layered package structure (core/calculator, core/recommender, core/glm_client, core/database); GLM system prompts are top-level constants in their respective modules; .env-driven configuration for endpoints and model names.*
Token Latency	*End-to-end Analyse-to-cards latency must be under 2 seconds; full GLM reasoning streamed to user under 20 seconds at p95.*	*Math core pre-computes options before any GLM call (instant cards); GLM responses are streamed (st.write_stream) so user sees first token in 1–3 seconds; non-streaming retry path on stream failure.*
Cost Efficiency	*Average GLM token cost per duty-manager session must stay under 1 sen (RM 0.01); never re-bill on page reload.*	*Prompt length bounded by design (one historical precedent, three pre-computed options, fixed system prompt); briefing cached in st.session_state once streamed; keyword fallback handles trivial extractions without GLM call.*

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026. [e.g. below]

a. *Real-time integration with live LRT operational systems (signalling, fare gates, CCTV) - the prototype produces recommendations only; physical schedule deployment remains in the existing OCC system.*
b. *Multi-line cascade modelling - each line is optimized independently in the MVP; cross-line passenger interchange impacts and fleet rebalancing are deferred to v2.*
c. *Multi-user concurrent editing, role-based access control, and SSO with the LRT operator's identity provider - deferred to v2 once the MVP is validated with a single duty manager.*

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing:*** *The MVP will be piloted with a single duty manager on one line (Kelana Jaya) for two weeks before being extended to Ampang and Sri Petaling. Within the pilot, an A/B test compares the AI-assisted decision (recommended option applied) against a control where the duty manager makes the call without seeing GLM's pick (only the three option cards). Metrics: time-to-decision, cost delta versus standard, and post-shift load-factor accuracy. Rollout proceeds to 100% only when AI-assisted decisions show statistically lower cost without service degradation.*

4.1.2. ***Emergency Rollback (ER) & Golden Release:*** *The previous 'Golden Release' is the manual standard timetable; the system can be rolled back to it instantly via the existing 'Reset to standard' button on every adjusted schedule. If a pattern of bad recommendations is detected (e.g. cost overruns or repeated overcrowding), the duty manager rolls back the specific affected day; if the issue is system-wide, the entire Streamlit process is taken offline and operations revert to the manual baseline with no data loss - saved schedules remain in SQLite for post-incident review.*

4.1.3. ***Priority Matrix:***

4.1.3.1. *P1 CRITICAL - GLM API unreachable for 5+ minutes or safety-override path fails (e.g. track_incident produces non-zero trains in any test): immediate rollback to manual baseline; freeze the affected build. P2 HIGH - GLM streaming fails for >2% of sessions in a 1-hour window: switch GLM client to non-streaming mode; investigate. P3 MEDIUM — cost delta drift >15% from predicted on adjusted days: trigger ridership baseline recalibration review. P4 LOW - UI cosmetic issues, non-blocking warnings: address in next sprint.*

© UMHackathon 2026

4.2. Monitoring:

4.2.1. *Application logs capture every GLM call with prompt token count, response latency, and any fallback path triggered. SQLite saved schedules form an implicit decision log - every adjustment is queryable by date, line, weather, and emergency type. A weekly review job (planned post-MVP) compares predicted load factor against actual ridership data to detect drift in baseline tables. Alerts: GLM API error rate >5% over 15 min triggers a Slack notification to the dev team; database write failures trigger immediate UI error and log capture for post-mortem.*

4.3. Assumptions: *State key operational & environmental conditions that you assumed to be valid while development of system and deployment.*

a. *The duty manager has stable internet access at the operations workstation throughout the shift, sufficient to maintain a streaming HTTPS connection to Z.AI's GLM API.*
b. *Standard ridership baselines (weekday, Saturday, Sunday hourly tables in calculator.py) are reasonably representative of current demand; significant divergence is detectable through the weekly review job once accumulated saved schedules provide ground truth.*

4.4. External Dependencies: *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*
Z.AI GLM API (ilmu-glm-5.1)	*Core LLM reasoning engine - handles unstructured situation extraction, multi-step reasoning over the three computed options, and structured-tail output (RECOMMENDATION: <choice>) for downstream parsing. Also generates the shift briefing for handover.*	*HIGH - if Z.AI is unreachable or rate-limited, the system's reasoning capability is lost. Mitigation: three-layer fallback (streaming retry → blocking call → deterministic math-based recommendation with explicit 'GLM unavailable' banner). Safety overrides are independent of GLM and continue to function.*
Streamlit Community Cloud (or container host)	*Hosts the application during the hackathon demo and pilot phase.*	*LOW - if the host is unavailable, the app can be run locally from any developer machine; SQLite is file-based*

© UMHackathon 2026

		so no separate database service is needed.

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. ***Project Timeline:*** *Day 1–2: Project planning, problem scoping, ridership baseline research, prompt engineering for situation extraction. Day 3–4: Math core (calculator.py) implementation: baselines, weather multipliers, event profiles, three-option computation. Day 5–6: GLM client and orchestrator: extraction, recommendation, streaming, fallbacks, system prompts. Day 7: SQLite persistence layer and saved-schedule retrieval. Day 8: Streamlit UI: input modes, schedule table with row styling, option cards, apply/save flow. Day 9: OCR pipeline, image compression, error handling, end-to-end QA on 10 scenarios. Day 10: Documentation (PRD, SAD, QATD), demo polish, deck preparation, dry-run.*

5.2. ***Team Members & Roles:*** *Backend / Math Lead: owns calculator.py, ridership baselines, event profiles, emergency recovery curve. GLM / Prompt Lead: owns glm_client.py, recommender.py, system prompts, fallback paths. Frontend / UX Lead: owns app.py, Streamlit layout, input modes, row styling, save / reset flow. Data / Persistence Lead: owns database.py, history.json schema, saved-schedule structure. PM / Documentation Lead: owns PRD, SAD, QATD, demo video, judging-criteria alignment, project timeline tracking.*

5.3. ***Recommendations:*** *Post-hackathon, the highest-leverage upgrades are: (1) replace SQLite with managed Postgres and migrate the JSON columns into normalized tables for queryable history; (2) move historical precedents from JSON-file lookup into a vector database (pgvector or FAISS) so retrieval scales beyond a few hundred records; (3) add a Redis cache for repeated identical extraction prompts (e.g. recurring weekly events); (4) instrument every saved schedule with actual post-shift ridership data to enable closed-loop baseline recalibration.*

© UMHackathon 2026